



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 4

Team No.: 2

Project Title: Linux Memory-Mapped File Management System

Team Members

Roll Number	Student Name	Signature
2520030081	S. Ashwanth Reddy	
2520030518	P. Naveen	
2520030280	CH. Nripendra	

1. Abstract

Memory-mapped file I/O is a mechanism provided by the Linux kernel that allows a file's contents to be mapped directly into a process's virtual address space, enabling applications to access file data using simple memory operations instead of traditional `read()/write()` system calls. Conventional file I/O involves repeated context switches and data copying between kernel and user space buffers, which becomes a performance bottleneck for large files or frequent access. This project proposes the design and implementation of a Linux Memory-Mapped File Management System that uses the `mmap()`, `munmap()`, and `msync()` system calls to provide efficient, low-overhead access to files. The system demonstrates core Operating Systems concepts such as virtual memory management, page tables, demand paging, and page fault handling, while addressing synchronization of in-memory changes back to persistent storage. The proposed solution includes utilities to map files into memory, perform sequential and random access, modify mapped regions, and safely synchronize or unmap them, along with performance comparisons against standard file I/O. The expected outcome is a functional command-line based file management tool that highlights the performance benefits of memory-mapped I/O and provides deeper insight into how the Linux kernel manages virtual memory and file-backed pages.

2. Problem Statement

Traditional file access methods in Linux, such as `read()` and `write()`, require data to be copied between kernel space and user space, introducing CPU overhead and reducing performance. This project addresses this issue by designing a memory-mapped file management system using the Linux `mmap()` system call. By mapping files directly into a process's address space, efficient file access is achieved using virtual memory and demand paging.

3. Objectives

1. To study and understand the Linux `mmap()` system call and its role in virtual memory management.
2. To design and implement a file management system that uses memory mapping for reading and writing files.
3. To implement synchronization of mapped memory changes to disk using `msync()` and safe unmapping using `munmap()`.
4. To compare the performance of memory-mapped file I/O with traditional `read()/write()` based file I/O.

4. Proposed Methodology

The system will be developed in C on a Linux/Ubuntu platform using standard POSIX APIs. The design begins with opening a file using `open()` and obtaining its file descriptor, followed by mapping the file into the process's virtual address space using `mmap()`. The mapped memory region is exposed to the user through command-line operations such as read, write, and search within the file. Modifications made to the mapped memory are flushed back to the underlying file using `msync()`, ensuring consistency between memory and disk. The application also handles unmapping using `munmap()` and proper closing of file descriptors to release system resources, with error handling for cases such as invalid file sizes, permission issues, and page faults.

Finally, the performance of the memory-mapped approach will be benchmarked against conventional `read()/write()` based file handling to evaluate improvements in speed and efficiency.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
<code>mmap()</code>	Maps a file's contents directly into the process's virtual address space
<code>munmap()</code>	Unmaps memory-mapped regions and releases virtual address space
File Descriptors (<code>open()</code> , <code>close()</code>)	Provides file handles required as the basis for <code>mmap()</code> operations
<code>msync()</code>	Synchronizes changes in mapped memory back to the disk file
Virtual Memory Management / Page Tables	Handles mapping of virtual pages to physical memory frames
Page Fault Handling	Manages demand paging when accessing mapped memory regions

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030081	S. Ashwanth Reddy	Design and implementation of the core <code>mmap()</code> -based file mapping module
2520030518	P. Naveen	Implementation of synchronization (<code>msync()</code>) and unmapping (<code>munmap()</code>) logic
2520030280	CH. Nripendra	Performance testing, benchmarking, and documentation/report preparation

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and testing environment for the system
C / C++	Implementation of the system using Linux system calls and POSIX APIs
GCC / Make	Compilation and build automation
GDB / Valgrind	Debugging and memory analysis
Git / GitHub	Version control and collaborative development

8. Expected Outcome

The project is expected to result in a working Linux-based command-line utility that allows users to map files into memory and perform read, write, and search operations efficiently using memory-mapped I/O. The system will correctly synchronize changes back to disk and safely release mapped memory and file resources. Performance evaluation is expected to show that memory-mapped file access performs better than traditional file I/O for large files and frequent access patterns, reducing CPU overhead caused by unnecessary data copies.

9. GitHub Repository

GitHub Repository Link: <https://github.com/ashwanthreddyy/OSSP>

Repository Created: _____ ☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Ms. Soumya Erukonda

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____ ☐ Yes ☐ No

Date: _____

Faculty Signature: _____